



Figure 4: Concept of a typical SSD

	Speed	Capacity	Price	Reliability	Power Usage	Quietness	Battery Life
HDD	★	★★★★	★★★	★★★★	★★	★★	★★
SSHD	★★	★★★	★	★★★★	★★	★★	★★
SSD	★★★★	★	★	★★★★	★★★	★★★	★★★

1GB: ★=Good ★★=Better ★★★=Best

Figure 5: A comparison of the features of HDDs, SSDs and SSHDs

that has to be made carefully. For example, computer gaming enthusiasts should choose the one with more flash – others can do a budget tally and go for moderate flash configuration.

If you do not care too much about performance, power, form factor and the weight of the device, or are on a really tight budget or simply need loads of space (1TB to 16TB), you will have to stick to HDD. This is a good choice for a home/office desktop PC and low-end laptops.

In simple words, the choice is a standard one that every computer user and engineer has to make — between capacity, cost and performance; and one wrong choice could drastically degrade your laptop/desktop experience. **END** 

References

- <http://www.seagate.com/in/en/tech-insights/adaptive-memory-in-sshd-master-tv/>
- <http://www.seagate.com/in/en/do-more/how-to-choose-between-hdd-storage-for-your-laptop-master-dm/>

By: Nilesh Govande

The author currently works at Seagate Development Centre, Pune. He has been working on different storage technologies for over 10 years and may be contacted at nileshgovande@gmail.com.

Continued from page 84...

```
<?php
//current use syntax
use MyLibrary\Abc\classTest1
use MyLibrary\Abc\classTest2
use MyLibrary\Abc\classTest3

//group use declaration syntax
use MyLibrary\Abc[classTest1, classTest2, classTest3];
```

Engine exceptions

The 'engine exceptions' feature comes with PHP 7 to facilitate the error handling process in code. It allows us to replace fatal and recoverable fatal errors by exceptions. Catchable errors can be displayed and defined actions can be taken. In case the exception is not caught, PHP returns the fatal error. The `\EngineException` objects do not extend the `\Exception` base class. This results in two kinds of exceptions—traditional and engine exceptions.

```
<?php
function MyFunction($obj)
{
    //code
}
MyFunction(null); //Fatal error
```

Considering the previous code, the code below shows how a fatal error is replaced by an `EngineException`:

```
try
{
    MyFunction(null);
}
catch(EngineException $ex)
{
    echo "Exception Caught";
}
```

Lists of deprecated items have been removed from PHP 7 -- for instance, ASP style tags (`<%=`, `<%`, `%>`) and script tags (`<script language="php">`), safe mode and its `ini` directives, and so on. MySQL extension has been deprecated since PHP 5.5, and replaced by the `mysqli` extension (`mysqli_*` functions). Likewise, the `ereg` extension has been deprecated since PHP 5.3, and replaced with the `PCRE` extension (`preg_*` functions). Do visit https://wiki.php.net/rfc/php_70 for more information. **END** 

References

- www.php.net
- www.zend.com/n/resources/php7_infographic

By: Krunal Patel

The author is assistant professor in an engineering institute. His area of interest is open source technologies. He can be reached at krunalpatel27@yahoo.co.in.